HDOC DAY-66

Write a recursive program to **multiply two square matrices**. Do not use iterative loops for the multiplication logic.

Algorithm

1. Start.
2. Read the order n of the square matrices.
3. Input the elements of matrices A and B.
4. Initialize resultant matrix C with zeros.
5. Call the recursive function multiply(A, B, C, n, 0, 0, 0).
6. In the recursive function:
    - If i == n, stop recursion.
    - If j == n, move to the next row.
    - If k == n, move to the next column.
    - Otherwise, calculate C[i][j] += A[i][k] * B[k][j].
    - Recursively call the function with k + 1.
7. Display the resultant matrix.
8. Stop.

• C Program:

```
#include <stdio.h>

#define MAX 10

void multiply(int A[MAX][MAX], int B[MAX][MAX],
              int C[MAX][MAX], int n,
              int i, int j, int k)
{
    if (i == n)
        return;

    if (j == n) {
        multiply(A, B, C, n, i + 1, 0, 0);
```

```
        return;
    }

    if (k == n) {
        multiply(A, B, C, n, i, j + 1, 0);
        return;
    }

    C[i][j] += A[i][k] * B[k][j];

    multiply(A, B, C, n, i, j, k + 1);
}

int main()
{
    int A[MAX][MAX], B[MAX][MAX];
    int C[MAX][MAX] = {0};
    int n, i, j;

    printf("Enter order of square matrices: ");
    scanf("%d", &n);

    if (n < 1 || n > MAX) {
        printf("Invalid matrix order.\n");
        return 1;
    }

    printf("Enter elements of first matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &A[i][j]);

    printf("Enter elements of second matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &B[i][j]);

    multiply(A, B, C, n, 0, 0, 0);

```

```
    printf("Resultant matrix:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++)
            printf("%d ", C[i][j]);
        printf("\n");
    }

    return 0;
}
```

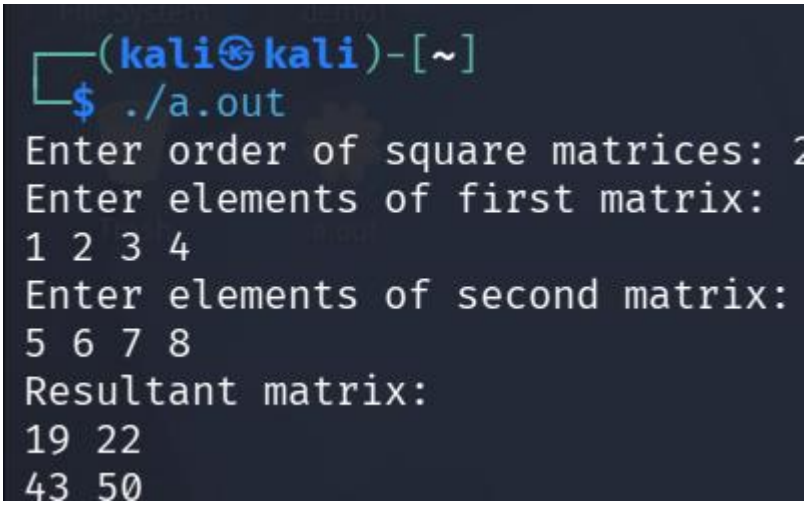
Test Case Table:

Test Case	Input matrices	Expected Output
1	A = [[1,2] , [3,4]], B = [[5,6] , [7,8]]	[[19 , 22] , [43 , 50]]
2	A = [[2]] , B = [[3]]	[[6]]
3	A = [[1 , 0] , [0 , 1]] , B = [[2 , 3] , [4 , 5]]	[[2 , 3] , [4 , 5]]
4	A = [[0 , 0] , [0 , 0]] , B = [[1 , 2] , [3 , 4]]	[[0 , 0] , [0 , 0]]
5	n = 0	Invalid matrix order

Compilation and Execution:

The following test cases has been compiled and executed successfully.

Test Case 1:



Test Case 2:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter order of square matrices: 1
Enter elements of first matrix:
2
Enter elements of second matrix:
3
Resultant matrix:
6
```

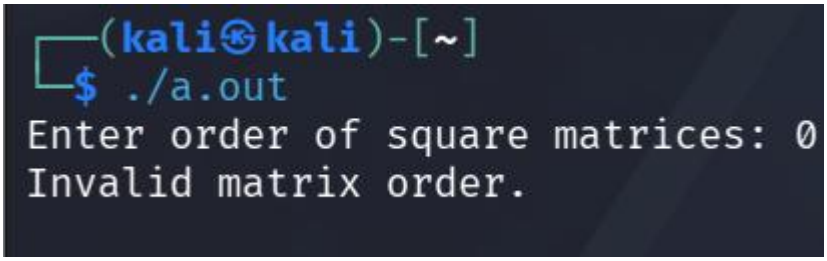
Test Case 3:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter order of square matrices: 2
Enter elements of first matrix:
1 0 0 1
Enter elements of second matrix:
2 3  4 5
Resultant matrix:
2 3
4 5
```

Test Case 4:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter order of square matrices: 2
Enter elements of first matrix:
0 0 0 0
Enter elements of second matrix:
1 2 3 4
Resultant matrix:
0 0
0 0
```

Test Case 5:


(kali㉿kali)-[~]
$ ./a.out
Enter order of square matrices: 0
Invalid matrix order.